Type Casting

Type Casting : Type Casting is the process of changing the value of one data type to another data type is called Type Casting.

- Casting in python is done using constructor functions, which are built-in functions used to convert one data type to another.
- **Common constructor Functions :**
  1. int() → Converts value to integer
  2. float() → Converts value to float
  3. str() → Converts value to string

1. **Integer :** int() is a built-in constructor function used to convert a value into an integer (whole number).it can take numbers, strings, or Boolean values and return an integer representation.
   Example :

```
x = int(1)
y = int(2.8)
z = int("3")
print(x,y,z)
```

   output : 1 2 3

   - After executing the program, if you're not sure whether it has been converted to an integer or not, you can check it using the "type()" function.

```
x = int(1)
y = int(2.8)
z = int("3")
print(type(x),type(y),type(z))
```

   output : <class 'int'> <class 'int'> <class 'int'>

2. **Float :** float() is a built-in constructor function used to convert a value into a floating point numbers(decimal number type).
   Example :

```
x = float(10)
y = float(4.8)
z = float("5")
print(x,y,z)
```

   output : 10.0 4.8 5.0

3. **String :** str() is a built-in constructor function used to convert any data type into a string (text format).

Example :

```
x = str("s1")
y = str(4.8)
z = str(3.0)
print(x,y,z)
```

output : s1 4.8 3.0

Code Challenge : Casting

Test your understanding of Python type casting by completing a small coding challenge

Instructions

Inside the editor, complete the following steps:

1. Create a variable x with the integer value **1**
2. Convert x to a float and store it in a
3. Convert x to a string and store it in b
4. Print a and b

Example :

```
x = int(1)
a = float(x)
b = str(x)
print(type(a),type(b))
print(a,b)
```

output : <class 'float'> <class 'str'>

1.0 1

Python Lists

List : A list is a built-in data types in python used to store multiple values in a single variable. It is ordered, changeable (mutable), and allows duplicate values.

Syntax :

list_name = [item1, item2, item3]

Example :

```
list_1 = ["one","two"]
print(list_1)
```

output : ['one', 'two']

Ordered : when we say that lists are ordered, it means that the items have a defined order, and that order will not change.

- If you add new items to a list , the new items will be placed at the end of the list.

Note: There are some list methods that will change the order, but in general: the order of the items will not change.

Changeable : The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Built-in Functios in List:

1. len() : len() is a built-in function used to return the total number of items (length) in an object such as a list, string, tuple, or dictionary.
   Syntax :
   len(object)

   Example :

```
ls = [1,2,3,4,5]
print(len(ls))
```

   output: 5

2. type() : type() is a built-in function used to identify and return the data type of a variable or value.
   Syntax :
   Type(object)
   Example :

```
x = (1)
y = ("hi")
z = (2.4)
print(type(x),type(y),type(z))
print(x,y,z)
```

   output : <class 'int'> <class 'str'> <class 'float'>
   1 hi 2.4

3. sum() : sum() is a built-in function used to add all the elements in an iterable (like a list or tuple) and return the total.
   Syntax :
   sum(variable)
   Example :

```
x = [1,2,3]
print(sum(x))
print(x)
```

output : 6
[1, 2, 3]

4. append() : append() is a built-in list method used to add a single element to the end of an existing list.it modifies the original list and increases its size by one.
   Syntax :
   list_name.append(item)
   Example :

```
x = [1,2,3]
print(x)
x.append(4)
print(x)
```

   output : [1, 2, 3]
   [1, 2, 3, 4]

5. insert() : insert() is a list method used to add an element at a specific index(Position) in the list without replacing existing elements.
   Syntax :
   list_name.insert(index,item)
   Example :

```
my_list = [1, 3]
my_list.insert(4, 4)
print(my_list)
```

   output : [1, 3, 4]

6. count() : count() is a list method used to return the total number of times a specified value appears in the list.
   Syntax :
   List_1.count(value)
   Example :

```
my_ls= [1,2,3,4,2,2,3]
print(my_ls.count(2))
```

   output : 3

7. pop() : pop() is a list method used to remove and return the element at a specified index from the list.
   Syntax :
   list_1.pop(index)
   - if no index is given, it removes the last item

Example :

```
my_ls= [1,2,3,4,2,2,3]
my_ls.pop(1)
print(my_ls)
```

output : [1, 3, 4, 2, 2, 3]

8. remove() : it is used to delete the first occurrence of a specified value from the list.
   Syntax :
   list_1.remove(value)
   Example :

```
my_ls= [1,2,3,4,2,2,3]
my_ls.remove(4)
print(my_ls)
```

output : [1, 2, 3, 2, 2, 3]

9. sort() : it is used to arrange the elements of a list in ascending(default) or descending order.
   Syntax :
   list_1.sort()
   Example :

```
my_ls= [1,2,3,4,2,2,3]
my_ls.sort()
print(my_ls)
```

output : [1, 2, 2, 2, 3, 3, 4]

10. Access : Accessing items means retrieving(getting)individual elements from a data structure(like a list, tuple or string) using their position or key.
    Syntax :
    list[index]
    - Index start from the 0(zero).

    Example :

```
x = [1,3,4,5]
print(x[0])
```

output : 1

11. Changing : it means updating or modifying the value of an existing element in a data structure like a list or dictionary.
    Syntax :
    list_1[index] = new_value

Example :

```
x = [1,3,4,5]
x[0] = 10
print(x)
```

output : [10, 3, 4, 5]

Code Challenge : List

Test your understanding of creating, accessing, changing, adding, and removing list items.

Instructions

Inside the editor, complete the following steps:

1. Create a list called colors with the values **"red"**, **"green"**, **"blue"**
2. Print the first item in the list
3. Change the second item to **"yellow"**
4. Add **"purple"** to the end of the list using append()
5. Remove **"red"** from the list using remove()
6. Print the list

Example :

```
colors = ["red","green","blue"]
print(colors[0])
colors[1]= "yellow"
print(colors)
colors.append("purpule")
colors.remove("red")
print(colors)
```

output : red

['red', 'yellow', 'blue']

['yellow', 'blue', 'purpule']

Sets

Set : A set is a built-in data type used to store multiple unique elements in a single variable. It is unordered, unindexed, and does not allow duplicate values.

Key Characteristics

- Unordered → items have no fixed position.
- Unindexed → Cannot access items using index.
- No duplicates → Each value appears only once.
- Mutable → You can add or remove items.

Syntax :

set_name ={item1,item2}

Example :

```
set_name = {1,3,4,5}
print(set_name)
```

output : {1, 3, 4, 5}

- Here we can see some built-in like
  1. len()
  2. type()
  3. add()
  4. update()
  5. remove()
  6. discard()
  7. pop()
  8. clear()

1. len() : len() is a built-in function used to return the number of elements in a set (or any iterable).
   Syntax :
   len(object)
   Example :

```
set_name = {1,3,4,5}
print(len(set_name))
```

   output : 4
2. type() : type() returns the data type of a variable or object.
   Syntax :
   type(object)

Example :

```
set_name = {1,3,4,5}
print(type(set_name))
```

output : <class 'set'>

3. add() : add() is a set method used to add a single element to a set.

   Syntax :

   set_1.add(element)

   Example:

```
set_name = {1,3,4,5}
set_name.add(2)
print(set_name)
```

output : {1, 2, 3, 4, 5}

4. update() : update() adds multiple elements from another iterable to the set.

   Syntax :

   set_1.update(iterable)

   Example :

```
set_name = {1,3}
set_name.update([2,6])
print(set_name)
```

output : {1, 2, 3, 6}

5. remove() : remove() is used to deletes a specific elements from the set.

   Syntax :

   set_1.remove(element)

   Example :

```
set_name = {1,2,3,2}
set_name.remove(2)
print(set_name)
```

output : {1, 3} #raises error if element is not found

6. discard() : it is used to removes an element without raising an error if it doesn't exist.

   Syntax :

   set_1.discard(element)

   Example :

```
set_name = {1,2,3,2}
set_name.discard(5)
print(set_name)
```

output : {1, 2, 3}

7. pop() : it is used to removes and returns a random element from the set.

Syntax :

set_1.pop()

Example :

```
set_name = {1,2,3,20,40}
set_name.pop()
print(set_name)
```

output : {2, 3, 20, 40}

8. clear() : it removes all elements from the set.

Syntax :

set_1.clear()

Example :

```
set_name = {1,2,3,20,40}
set_name.clear()
print(set_name)
```

output: set() # set becomes empty

Code Challenge : Sets

Test your understanding of creating, adding, removing, and checking set items.

Instructions

Inside the editor, complete the following steps:

1. Create a set called colors with the values **"red"**, **"green"**, **"blue"**
2. Print the set
3. Add **"yellow"** to the set using add()
4. Remove **"green"** from the set using discard()
5. Print the number of items using len()

Example :

```
colors = {"red","green","blue"}
print(colors)
colors.add("yellow")
colors.discard("green")
print(colors)
print(len(colors))
```

output : {'red', 'blue', 'green'}

{'yellow', 'red', 'blue'}

3